Write a Custom Event Handler Using the WSO2 Identity Server Eventing Framework

Introduction

WSO2 Identity Server publishes an **event** before and after many operations, for example:

Event	Published
PRE_AUTHENTICATION / POST_AUTHENTICATION	Before and after a user's password is checked
PRE_ADD_USER / POST_ADD_USER	Before and after a user is added
PRE_SET_USER_CLAIMS / POST_SET_USER_CLAIMS	Before and after user attributes are changed
PRE_UPDATE_CREDENTIAL / POST_UPDATE_CREDENTIAL	Before and after a password is changed

An **event handler** is a class that subscribes to events and runs your code when they are published. When a user is added, WSO2 Identity Server:

1. Publishes **PRE_ADD_USER**. Every handler subscribed to it runs.
2. Adds the user to the user store.
3. Publishes **POST_ADD_USER**. Every handler subscribed to it runs.

In this lab you build, deploy and test the WSO2 sample **custom-identity-event-handler**. It writes a line to the server log before and after a user is added. In a real project you would

CC BY 4.0

put your own logic there, for example sending the new user to another system.

Time: about 30 minutes.

Prerequisites

1. WSO2 Identity Server 7.3.0 (`<IS_HOME>` below) and JDK 21 with `JAVA_HOME` set.
2. Apache Maven 3.6 or later, and Git. The machine needs internet access to `https://maven.wso2.org` and Maven Central, because the build downloads the WSO2 libraries.

Step 1: Get the sample

1. Clone the WSO2 samples repository (or download it as a ZIP from GitHub and extract it). The folder is `<SAMPLE_IS>` below:

   ```
   git clone https://github.com/wso2/samples-is.git
   ```

2. The sample is in `<SAMPLE_IS>/event-handler/custom-identity-event-handler`. It has three classes:

   ```
   src/main/java/org/wso2/carbon/identity/customhandler/handler/
   UserRegistrationCustomEventHandler.java
   src/main/java/org/wso2/carbon/identity/customhandler/internal/
   UserRegistrationCustomEventHandlerComponent.java
   src/main/java/org/wso2/carbon/identity/customhandler/internal/
   UserRegistrationCustomEventHandlerDataHolder.java
   ```

Step 2: Read the code

You do not need to change the Java code. These excerpts show the important parts (imports are left out).

CC BY 4.0

The event handler

The handler extends `AbstractEventHandler`. `getName()` returns the name you use in `deployment.toml` (Step 4). `getPriority()` sets the order when several handlers subscribe to the same event: a lower number runs first.

```
package org.wso2.carbon.identity.customhandler.handler;

public class UserRegistrationCustomEventHandler extends
AbstractEventHandler {

    private static final Log log =
LogFactory.getLog(UserRegistrationCustomEventHandler.class);

    @Override
    public String getName() {
        return "customUserRegistration";
    }

    @Override
    public int getPriority(MessageContext messageContext) {
        return 250;
    }
```

`handleEvent()` runs for every event the handler is subscribed to. The event name tells you which one it is, and `getEventProperties()` gives the details, such as the user name and the tenant:

```
    @Override
    public void handleEvent(Event event) {

        if
(IdentityEventConstants.Event.PRE_ADD_USER.equals(event.getEventName())) {
            String tenantDomain = (String) event.getEventProperties()
                    .get(IdentityEventConstants.EventProperty.TENANT_DOMAI
N);
            String username = (String) event.getEventProperties()
```

CC BY 4.0

```
                    .get(IdentityEventConstants.EventProperty.USER_NAME);
            log.info("Handling the event before adding user: " + username
                    + " in tenant domain: " + tenantDomain);
        }

        if
(IdentityEventConstants.Event.POST_ADD_USER.equals(event.getEventName()))
{
            // same as above, with "after adding user"
        }
    }
}
```

The OSGi component

When the bundle starts, this component registers the handler with the eventing framework. The `@Reference` makes it wait for the user management service (RealmService):

```
package org.wso2.carbon.identity.customhandler.internal;

@Component(
        name =
"org.wso2.carbon.identity.customhandler.internal.UserRegistrationCustomEve
ntHandlerComponent",
        immediate = true)
public class UserRegistrationCustomEventHandlerComponent {

    @Activate
    protected void activate(ComponentContext context) {
        try {
            BundleContext bundleContext = context.getBundleContext();

bundleContext.registerService(AbstractEventHandler.class.getName(),
                    new UserRegistrationCustomEventHandler(), null);
        } catch (Exception e) {
            log.error("Error while activating custom User selfRegistration
handler component.", e);
```

CC BY 4.0

```
        }
    }

    @Reference(name = "RealmService",
            service =
org.wso2.carbon.user.core.service.RealmService.class,
            cardinality = ReferenceCardinality.MANDATORY,
            policy = ReferencePolicy.DYNAMIC,
            unbind = "unsetRealmService")
    protected void setRealmService(RealmService realmService) { ... }
    ...
}
```

Step 3: Build the bundle

1. **Fix the Java version in the sample first.** Open `<SAMPLE_IS>/event-handler/custom-identity-event-handler/pom.xml` and find the `maven-compiler-plugin`. It sets Java 1.7 with an old plugin version. JDK 17 and JDK 21 cannot compile for Java 1.7 (the build stops with `Source option 7 is no longer supported. Use 8 or later.`). Change it to:

   ```
   <plugin>
       <groupId>org.apache.maven.plugins</groupId>
       <artifactId>maven-compiler-plugin</artifactId>
       <version>3.8.1</version>
       <configuration>
           <source>1.8</source>
           <target>1.8</target>
       </configuration>
   </plugin>
   ```

2. Build the sample:

   ```
   cd <SAMPLE_IS>/event-handler/custom-identity-event-handler
   mvn clean install
   ```

3. When the build finishes with `BUILD SUCCESS`, the bundle is in the `target` folder:

CC BY 4.0

`org.wso2.carbon.identity.customhandler-<version>.jar` (for example `org.wso2.carbon.identity.customhandler-4.6.5-SNAPSHOT.jar`).

Step 4: Deploy the handler

1. Stop WSO2 Identity Server if it is running.
2. Copy the JAR from Step 3 into `<IS_HOME>/repository/components/dropins`.
3. Open `<IS_HOME>/repository/conf/deployment.toml` and add at the end:

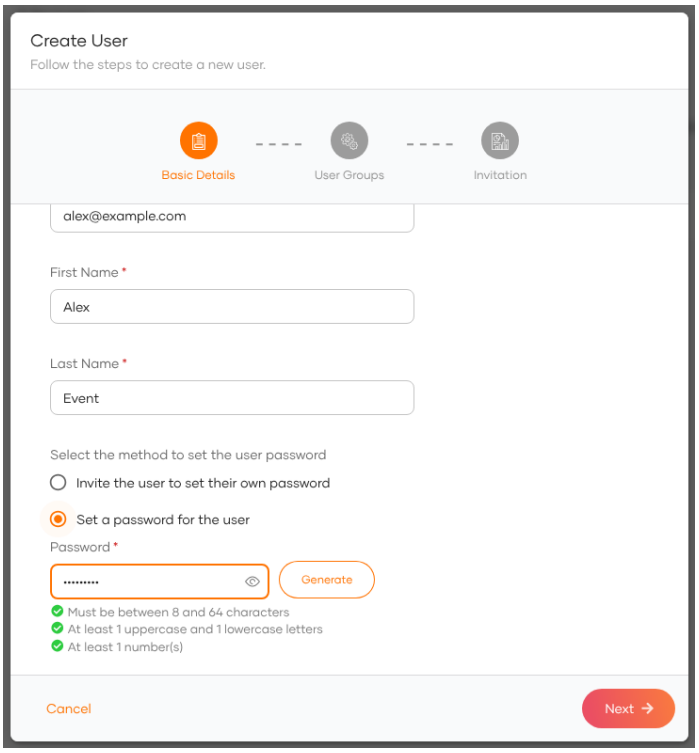
   ```
   [[event_handler]]
   name = "customUserRegistration"
   subscriptions = ["PRE_ADD_USER","POST_ADD_USER"]
   ```

   `name` must be exactly the value that `getName()` returns. `subscriptions` lists the events the handler receives. Without this section the handler is deployed but never called.
4. Start WSO2 Identity Server: `wso2server.bat` (Windows) or `sh wso2server.sh` (Linux and macOS) from `<IS_HOME>/bin`. Wait for `WSO2 Carbon started`. The sample does not print anything at startup.

Try It

1. Log in to the Console (`https://localhost:9443/console`) as `admin` / `admin`.
2. Go to **User Management > Users > Add User > Single User**. Enter the username `alex`, the email `alex@example.com`, the first name `Alex` and the last name `Event`. Select **Set a password for the user**, enter `Wso2@1234` and finish the wizard.

CC BY 4.0


Create User
Follow the steps to create a new user.
Basic Details
User Groups
Invitation
alex@example.com
First Name *
Alex
Last Name *
Event
Select the method to set the user password
Invite the user to set their own password
Set a password for the user
Password *
Generate
Must be between 8 and 64 characters
At least 1 uppercase and 1 lowercase letters
At least 1 number(s)
Cancel
Next

CC BY 4.0

3. Check the server log (the terminal where WSO2 Identity Server runs, or `<IS_HOME>/repository/logs/wso2carbon.log`). Your handler ran once before and once after the user was added:

```
[2026-11-14 17:07:21,010]  INFO {...UserRegistrationCustomEventHandler} - Handling the event
    before adding user: alex in tenant domain: carbon.super
[2026-11-14 17:07:21,226]  INFO {...UserRegistrationCustomEventHandler} - Handling the event
    after adding user: alex in tenant domain: carbon.super
```

CC BY 4.0

The events are published by the user management layer, not by the Console, so the handler is also called when users are added in other ways, for example through self-registration or the SCIM 2.0 API.

Change the subscriptions

1. Stop WSO2 Identity Server, change `subscriptions` to `["POST_ADD_USER"]` and start it again.
2. Add another user (for example `bella`). Only the **after adding user** line is printed: `deployment.toml` controls which events reach the handler, without changing the code.
3. To see what the configuration does, remove the whole `[[event_handler]]` section, restart and add a third user. Nothing is printed: the handler is deployed, but it is not subscribed to any event. Add the section back when you are done.

Troubleshooting

Problem	Cause and fix
Source option 7 is no longer supported. Use 8 or later. (during mvn clean install)	The sample's pom.xml still sets Java 1.7. Change the maven-compiler-plugin as shown in Step 3.
mvn clean install fails with Could not resolve dependencies / Non-resolvable parent POM	The machine cannot reach https://maven.wso2.org or Maven Central. Check the internet connection or the proxy settings in <MAVEN_HOME>/conf/settings.xml.
No "Handling the event" lines when a user is added	The [[event_handler]] section is missing, the name is not exactly customUserRegistration, the event is not in subscriptions, or the server was not restarted. Also check that the JAR is in dropins (not lib).
Only one of the two lines is printed	Only one event is listed in subscriptions.

CC BY 4.0

CC BY 4.0